INTERNSHIP PROJECT · TÜRKSAT AVIATION AND COMMUNICATION

Smart School Information System

Week 1 — Problem Definition, Architecture and Data Model

PREPARED BY	Bera Ergin — s29315
INSTITUTION	PJAIT — 7th Semester
SCOPE	Version 1 of the 12-week internship program

Problem Definition

Why is data access difficult in schools?

1

Querying data requires SQL knowledge

Administrators, teachers, and parents depend on the IT team or pre-built reports to access student data.

2

Access rules are enforced manually

Rules like “teachers should only see their own class” are hard-coded into report templates — not centralized or auditable.

3

No early-warning mechanism

Students with rising absences or falling grades are only noticed once the situation has already worsened.

Solution Vision

A rule-based, explainable AI Agent

The user asks a question in their own words, with no SQL required. The system understands the intent, generates the query, passes it through a database-level role-based access rule, and returns the result together with its reasoning.

- **AI Agent**
- **Database-Level Access Control**
- **Explainable AI**
- **Row-Level Security**

Teacher

“Show students whose absences exceeded 5 days this month.”

→ Runs only for class 5-A; result returned with reasoning.

Parent

“How did my child’s math grade change compared to last month?”

→ Answered using only that child’s data.

Roles and Permission Boundaries

Each role sees only the data within its own scope

1

Principal

All student and school data

2

Teacher

Only their own class

3

Parent

Only their own child

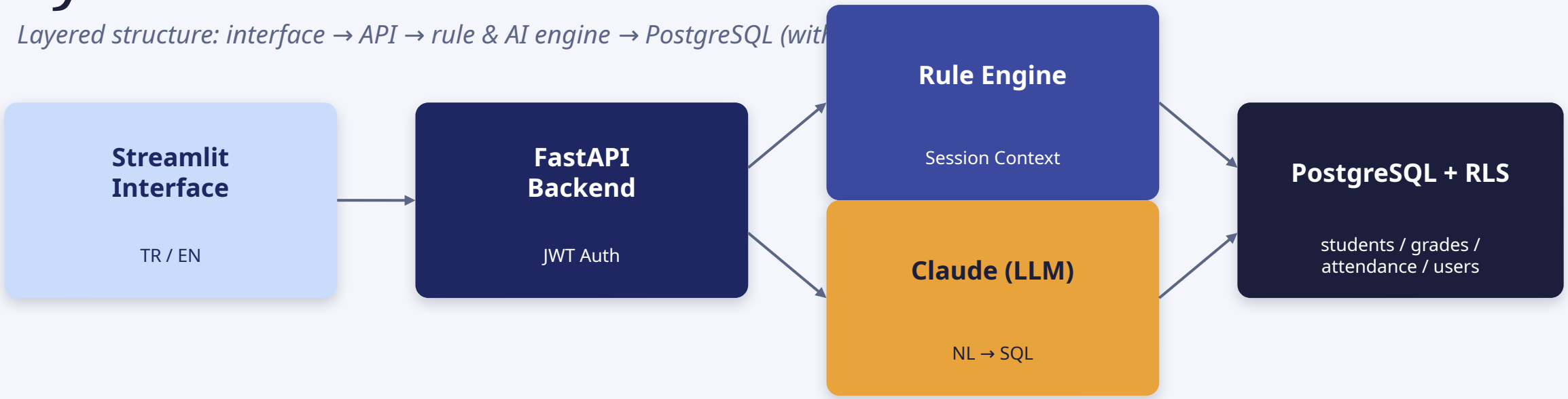
4

Student

Only their own information

System Architecture

Layered structure: interface → API → rule & AI engine → PostgreSQL (with



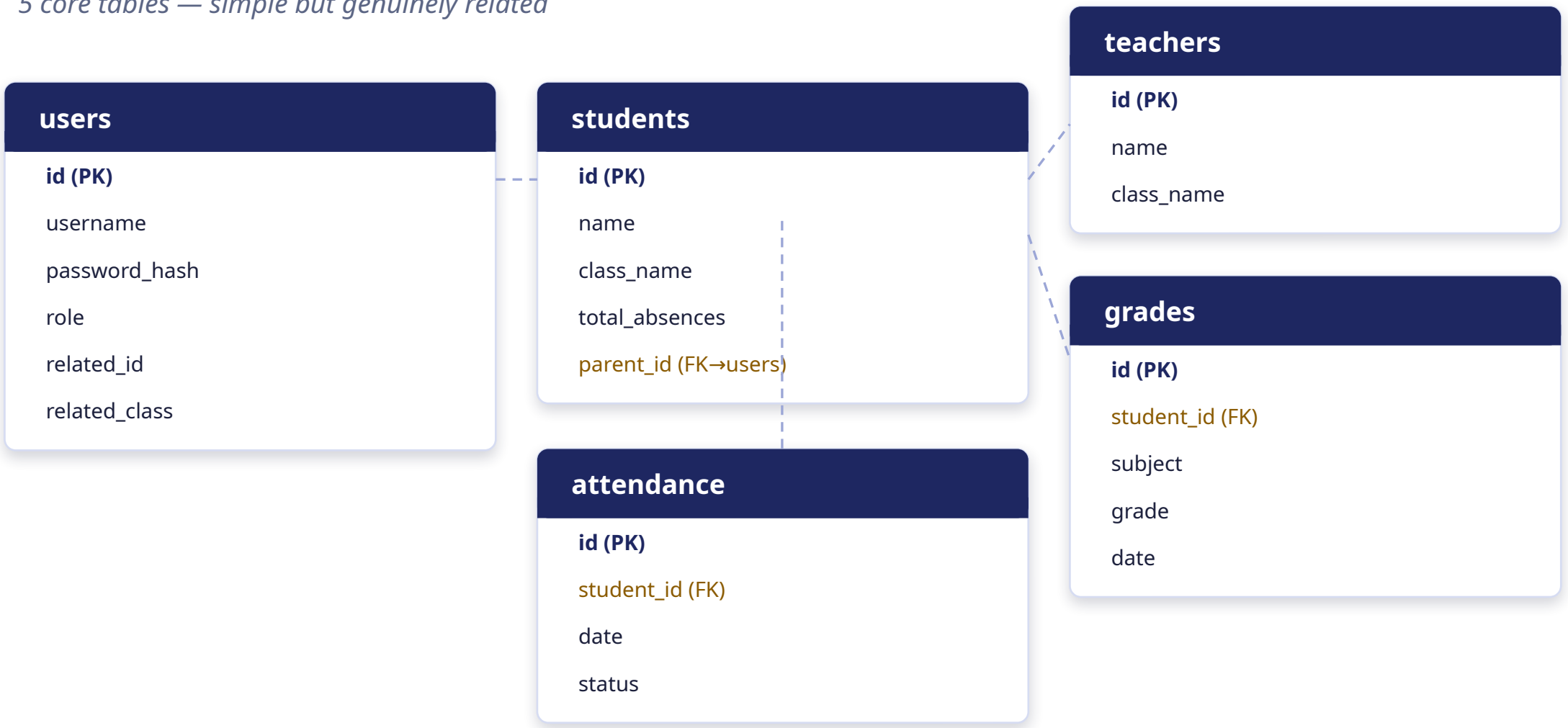
The difference: the role-based filter isn't added to the SQL text — PostgreSQL itself filters rows based on the context (role, class, student id) declared on the user's session.

Week 1 scope

This week only sets up the database and RLS policies — the API, rule engine, and LLM integration will be implemented in Weeks 2–5.

Data Model

5 core tables — simple but genuinely related



Why PostgreSQL + RLS from Day One

The access rule isn't in the SQL text — it's built into the database engine

Alternative — filter in app code

Manually adding a WHERE clause to every query, or parsing and modifying the generated SQL text afterward.

Becomes fragile as the SQL's shape changes (aliases, subqueries, JOINs).

This project — Row-Level Security

The session declares “this user's role is X, class is Y”; PostgreSQL filters rows automatically on EVERY query, whatever shape it takes.

Technology choices

Backend: FastAPI, SQLAlchemy, python-jose, passlib · Database: PostgreSQL 16 + Row-Level Security · Interface: Streamlit, TR/EN · AI: Anthropic Claude

Live Proof of RLS

No API, no application code — just the database

```
SET LOCAL app.role = 'teacher';
SET LOCAL app.class = '9-A';
SELECT COUNT(*) FROM students;
→ 4
```

```
SET LOCAL app.role = 'principal';
SELECT COUNT(*) FROM students;
→ 12
```

The same query (SELECT COUNT(*) FROM students), the same table — the only thing that changes is the session's context. This separation is built this week because every layer built on top of it (the API, the LLM) will benefit from this guarantee — whatever shape the SQL takes, the filter can never be skipped.

Week 1 — Scope and Acceptance Criteria

Delivered this week

- ✓ PostgreSQL (Docker container)
- ✓ 5-table data model (ER schema)
- ✓ init_db.py + seed_mock.py + mock data
- ✓ enable_rls.py — RLS policies for 3 tables
- ✓ test_rls.py — proof RLS works independent of the API
- ✓ Architecture diagram and role/permission matrix

Acceptance criteria

1. PostgreSQL comes up with docker run
2. init_db.py + seed_mock.py run without errors
3. enable_rls.py enables RLS on all 3 tables
4. test_rls.py: teacher→4, principal→12, parent→1
5. README explains the PostgreSQL + RLS rationale

Next Step — Week 2

FastAPI Skeleton and Authentication

- FastAPI project skeleton and router structure
- JWT-based login endpoint
- Password hashing with bcrypt
- Testing the 4 roles (principal/teacher/parent/student) with sample users

Thank you